



# Java et la Programmation Orientée Objet



Houssem MAHMOUDI  
houssem.mahmoudi@ensicaen.fr

Ensicaen©2025 - HM



## Chapitre 9 - Les interfaces graphiques AWT & Swing

1. Introduction à la Programmation Graphique en Java
2. La bibliothèque AWT
3. Gestion des Événements
4. La bibliothèque Swing
5. Connexion à la base de données avec JDBC

Ensicaen©2025 - HM

2

## I. Introduction à la Programmation Graphique en Java

- Le langage Java permet de créer des interfaces graphiques « GUI : **G**raphical **U**ser **I**nterface » appelée aussi IHM « **I**nterface **H**omme **M**achine ».
- A la différence du mode console qui est basé sur une interaction et affichage textuels, le mode graphique permet aux utilisateurs d'interagir via des éléments visuels (clic, clavier,...).
- Le langage Java propose différentes bibliothèques pour créer des GUI :
  - AWT
  - Swing
  - JavaFx
  - JFace

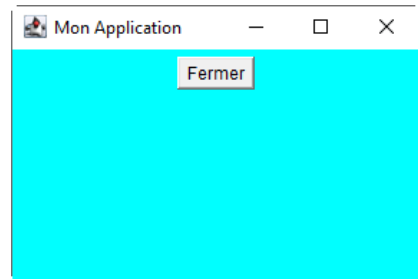
## II. La bibliothèque AWT

- **AWT** (**A**bstract **W**indow **T**oolkit) est la première bibliothèque des composants graphiques créée pour Java 1.0
- Les classes AWT se trouvent dans le package « **java.awt** ».
- Les composants graphiques AWT sont des composants lourds, car ils dépendent du système d'exploitation. (écrites en C et C++).
- Les interfaces graphiques n'utilisent plus la bibliothèque AWT, car elle est pauvre, mais certains concepts se maintiennent :
  - Classes de mise en forme (**Layout**).
  - La programmation événementielle.

## II. La bibliothèque AWT

### ▪ Exemple :

```
import java.awt.*;
public class AppGUISimple {
    public static void main(String[] args) {
        Frame f = new Frame("Mon Application");
        Button btn = new Button("Fermer");
        f.setSize(300, 200);
        f.setVisible(true);
        f.setBackground(Color.CYAN);
        f.setLayout(new FlowLayout());
        f.add(btn);
        btn.addActionListener(e -> f.dispose());
    }
}
```



## II. La bibliothèque AWT

- La classe java.awt.**Graphics** est utilisée pour gérer un contexte graphique (lignes, des rectangles, des cercles, du texte et des images).
- Elle est souvent utilisée avec **AWT** (*java.awt*) et **Swing** (*javax.swing*).
- La classe Graphics est une classe abstraite.
- Un objet Graphic est automatiquement fourni par le système :
  - Transmis en argument aux méthodes : public void **update**(Graphics g) et public void **paint**(Graphics g)
  - peut être également créé par les méthodes getGraphics() ou create()

### ↳ Exemple :

```
public void paint(Graphics g) {
    g.drawLine(50, 50, 200, 200);
}
```

## II. La bibliothèque AWT

### Exemples de méthodes de la classe java.awt.Graphics

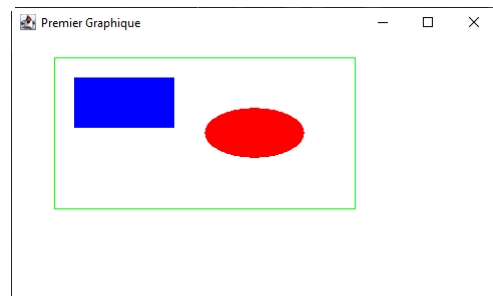
- public void **drawLine**( int x1, int y1, int x2, int y2 )
- public void **drawPolygon**( int xPoints[], int yPoints[], int nPoints )
- public void **drawRect**( int x, int y, int width, int height )
- public void **fillOval**( int x, int y, int width, int height )
- public void **fillRect**( int x, int y, int width, int height )
- public void **setColor** (Color c)

## II. La bibliothèque AWT

```
import java.awt.Color;
import java.awt.Frame;
import java.awt.Graphics;

class MonGraphique extends Frame {
    public MonGraphique() {
        setTitle("Premier Graphique");
        setSize(500, 300);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.fillRect(70, 70, 100, 50);
        g.setColor(Color.RED);
        g.fillOval(200, 100, 100, 50);
        g.setColor(Color.GREEN);
        g.drawLine(50, 50, 350, 50);
        g.drawLine(50, 50, 50, 200);
        g.drawLine(50, 200, 350, 200);
        g.drawLine(350, 50, 350, 200);
    }
}

public class TestAWT {
    public static void main(String[] args) {
        new MonGraphique();
    }
}
```



## II. La bibliothèque AWT

### La classe java.awt.Color

- La gestion des couleurs est basée sur un modèle à 24 bits.
- Une couleur est définie par ses composantes RGB
- Exemple : `Color BleuPale = new Color (0,0,80) ;`
- Certaines couleurs ont un nom symbolique (membres statiques)
- Exemples :
  - `Color.white`                255,255,255
  - `Color.red`                255,0,0
  - `Color.orange`            255,200,0
- Méthodes de gestion de couleurs appartenant à la classe `java.awt.Graphics` :
  - `public void setColor (Color c )`
  - `public Color getColor()`

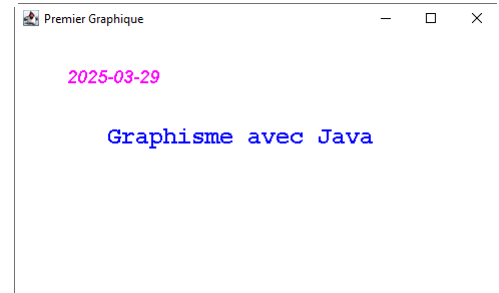
## II. La bibliothèque AWT

### La classe java.awt.Font

- Permet de définir des objets représentants des polices de caractères caractérisées par leur nom, leur style et leur taille de police.
- Exemple : `Font fo = new Font ("Monospaced", font.BOLD, 24) ;`
- En fonction de la plate forme utilisée, d'autres polices peuvent être disponibles.
- Styles disponibles:
  - `Font.BOLD`
  - `Font.ITALIC`
  - `Font.PLAIN`

## II. La bibliothèque AWT

```
import java.awt.Color;
import java.awt.Font;
import java.awt.Frame;
import java.time.LocalDate;
import java.awt.Graphics;
class MonGraphique extends Frame {
    public MonGraphique() {
        setTitle("Texte Graphique");
        setSize(500, 300); setVisible(true);
    }
    public void paint(Graphics g) {
        Font fDate = new Font("SansSerif", Font.ITALIC, 18) ;
        Font fTitre = new Font("Monospaced", Font.BOLD, 24) ;
        g.setFont(fDate);
        g.setColor(Color.MAGENTA);
        LocalDate today = LocalDate.now();
        g.drawString(today.toString(), 60, 80);
        g.setColor(Color.BLUE);
        g.setFont(fTitre);
        g.drawString("Graphisme avec Java", 100, 140);
    }
}
public class TestAWT {
    public static void main(String[] args) {
        new MonGraphique();
    }
}
```



## II. La gestion d'événements

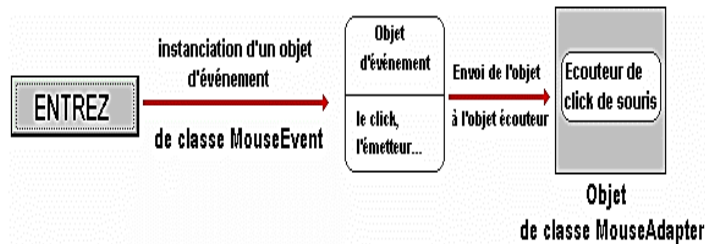
- En Java, la gestion des événements (Event Handling) est un mécanisme permettant aux objets d'interagir en réponse à des actions de l'utilisateur détectée par le programme.
- Un objet source génère un événement et un écouteur (listener) le traite.
- Java fournit des classes et interfaces spécifiques dans le package `java.awt.event` et `javax.swing.event` pour gérer ces interactions.
- Exemple d'événements :
  - Clic de souris,
  - presse d'une touche de clavier,
  - ...

## II. La gestion d'évènements

- Exemple d'un évènement :

**Clic sur un bouton**

- ↳ Création d'un objet évènement de type `ActionEvent`
- ↳ Envoi cet objet à un Ecouteur (Listener)
- ↳ L'écouteur doit exécuter (réagir) au clic.



## III. La bibliothèque Swing

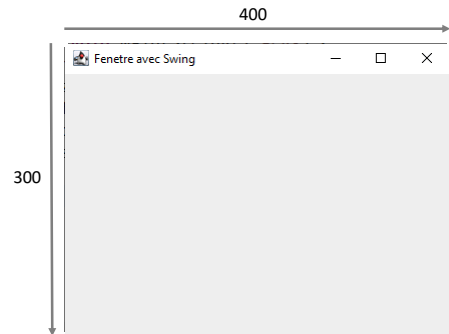
- La bibliothèque Swing fait partie du package « **javax.swing** »
- Forme une nouvelle hiérarchie parallèle à celle de l'AWT.
- Tous ses composants sont écrits en pur Java.
- Exemples de classes :

|                                       |                                                               |
|---------------------------------------|---------------------------------------------------------------|
| <code>javax.swing.JFrame</code>       | Classe permettant de créer et manipuler une fenêtre           |
| <code>javax.swing.JPanel</code>       | Classe permettant de créer et manipuler les panneaux.         |
| <code>javax.swing.JButton</code>      | Classe et interfaces de bases pour manipuler les boutons      |
| <code>javax.swing.border</code>       | Classe permet de manipuler les bordures                       |
| <code>javax.swing.colorchooser</code> | Classe définissant un composant pour la sélection de couleurs |

## 1. Fenêtre graphique – JFrame

- La création d'une fenêtre graphique se fait avec la classe **JFrame** du package **javax.swing**.
- Constructeurs :
  - `JFrame fen = new JFrame();`
  - `JFrame fen = new JFrame("titre");`

```
import javax.swing.JFrame;
public class Fenetre {
    public static void main(String[] args) {
        JFrame fen = new JFrame("Fenetre avec Swing");
        fen.setSize(400, 300);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setLocationRelativeTo(null);
        fen.setVisible(true);
    }
}
```

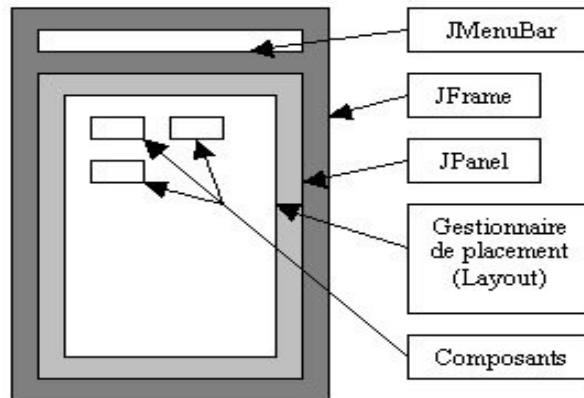


## 1. Fenêtre graphique – JFrame

- Les principaux méthodes de la classe **JFrame** sont :
  - `setTitle(String title)` ➤ Définit le titre de la fenêtre.
  - `setSize(int width, int height)` ➤ Définit la taille de la fenêtre.
  - `setBounds(int x, int y, int width, int height)` ➤ Définit la position et la taille de la fenêtre.
  - `setLocation(int x, int y)` ➤ Positionne la fenêtre à des coordonnées spécifiques.
  - `setResizable(boolean resizable)` ➤ Permet ou interdit le redimensionnement de la fenêtre.
  - `setVisible(boolean visible)` ➤ Affiche ou cache la fenêtre.
  - `setDefaultCloseOperation(int operation)` ➤ Définit le comportement lors de la fermeture.
  - `add(Component comp)` ➤ Ajoute un composant à la fenêtre.
  - `remove(Component comp)` ➤ Supprime un composant de la fenêtre.
  - `addWindowListener(WindowListener l)` ➤ Ajoute un écouteur d'événements à la fenêtre.
  - `pack()` ➤ Ajuste la taille de la fenêtre en fonction des composants.

## 2. Notion de Panneau – JPanel

- Un panneau est un conteneur destiné à contenir des composants graphique (boutons, zones de texte, liste déroulante,...).
- Un panneau est une sorte de « sous-fenêtre » sans titre ni bordure.
- La classe JPanel se trouve dans le package javax.swing.JPanel



Ensicaen©2025 - HM

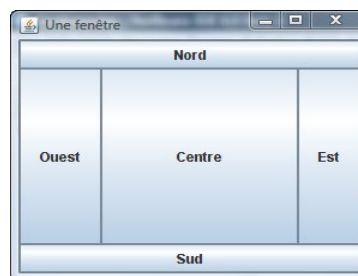
17

## 3. Gestionnaire de mise en forme – Layout

- Le gestionnaire de mise en forme (*LayoutManager*) est responsable de positionner les objets graphiques dans un conteneur (*JPanel*).

- a. **Gestionnaire BorderLayout** : ce gestionnaire dispose les composants dans l'un des quatre bords du conteneur et au centre.

- BorderLayout.NORTH
- BorderLayout.SOUTH
- BorderLayout.EAST
- BorderLayout.WEST
- BorderLayout.CENTER



- Le gestionnaire de mise en forme de JFrame par défaut est BorderLayout
- **Exemple :**

```
BorderLayout BL = new BorderLayout(15,10);
pan.setLayout(BL);
pan.add(ZonText, BorderLayout.NORTH);
pan.add(Bouton, BorderLayout.SOUTH);
```

Ensicaen©2025 - HM

18

### 3. Gestionnaire de mise en forme – Layout

#### b. Gestionnaire FlowLayout :

- Ce gestionnaire dispose les composants les uns à la suite des autres sur une même ligne.
- Si la ligne ne possède pas suffisamment d'espace, l'affichage se poursuit sur la ligne suivante.
- L'espacement par défaut entre les composants sur une même ligne est de 5 pixels.
- La disposition peut être :
  - FlowLayout.LEFT
  - FlowLayout.CENTER
  - FlowLayout.RIGHT
- **Exemple :**

```
JPanel pan = new JPanel();
FlowLayout FL = new FlowLayout (FlowLayout.CENTER, 10, 15);
pan.setLayout(FL);
pan.add(ZonText);
pan.add(Bouton);
```

### 3. Gestionnaire de mise en forme – Layout

#### c. Gestionnaire GridLayout :

- Ce gestionnaire permet de disposer les composants selon une grille régulière, chaque composant ayant la même taille.
- Les composants sont ajoutés de gauche à droite et de haut en bas dans l'ordre d'ajout.
- La disposition peut être :
  - new GridLayout (4, 3) ; ➡ Construction d'une grille de 5 lignes et 4 colonnes.
  - new GridLayout (4, 3, 15, 10) ; ➡ Comme la précédente avec espacement horizontal de 15 pixels et espacement vertical de 10 pixels.
- **Exemple :**

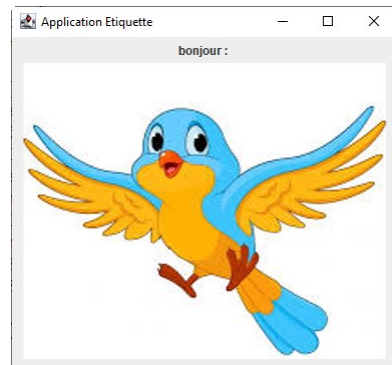


## 4. Les Etiquettes – JLabel

- La classe `javax.swing.JLabel` est un composant graphique permet d'afficher un texte non modifiable par l'utilisateur, mais que par programmation.
- Les constructeurs de `JLabel` sont :
  - `new JLabel("Bonjour");` ➔ Label avec le texte Bonjour.
  - `new JLabel(Icon image);` ➔ Affichage d'une image
- Principaux méthodes :
  - `void setText(String t)` : Affecte un texte au label.
  - `String getText()` : Retourne le texte du label.
  - `void setFont(Font f)` : Applique une police au label.

## 4. Les Etiquettes – JLabel

```
import java.awt.BorderLayout;
import javax.swing.Icon;
import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
class Monlabel {
    public Monlabel() {
        JFrame fen = new JFrame("Application Etiquette");
        fen.setSize(400, 370);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel pan = new JPanel();
        JLabel etq1 = new JLabel("bonjour : ");
        Icon ic = new ImageIcon(getClass().getResource("oiseau.png"));
        JLabel etq2 = new JLabel(ic);
        pan.add(etq1, BorderLayout.NORTH);
        pan.add(etq2, BorderLayout.SOUTH);
        fen.setLocationRelativeTo(null);
        fen.add(pan); fen.setResizable(false);
        fen.setVisible(true);
    }
}
public class Etiquette {
    public static void main(String[] pm) {
        Monlabel lab = new Monlabel();
    }
}
```



➔ L'image doit se être avec le fichier .class

## 5. Les champs texte – JTextField

- La classe `javax.swing.JTextField` est un composant rectangulaire avec bordure dans laquelle l'utilisateur peut entrer ou modifier un texte d'une seule ligne.
- Les **constructeurs** de `JTextField` sont :
  - `new JTextField(30);` ➡ champ de taille 30, initialement vide.
  - `new JTextField("mon texte initiale", 50);` ➡ champ de taille 50 contenant au départ le texte "mon texte initiale"
- Les principaux **méthodes** sont :
  - `String getText()` : Récupère l'information dans la zone de texte .
  - `void setEditable(false)` : Désactiver la modification du texte dans le champ de texte .
  - `void setHorizontalAlignment(JTextField.CENTER)` : Préciser l'alignement de texte dans le champ.
  - `setToolTipText("merci de saisir votre nom")` : Créer des infos bulle lors du pointage de souris.
  - `setBackground(Color.red)` : Ajouter un arrière-plan au champ de texte.

## 5. Les champs texte – JTextField

### Evènement sur les champs de texte :

- Un `JTextField` peut réagir aux plusieurs évènement :
- Evènement de catégories **Focus** : on peut attacher un écouteur directement au composant. L'écouteur implémente l'interface **FocusListener** qui comporte deux méthodes abstraites :
  - ❖ `public void focusGained(FocusEvent e)` : évènement se produit si le champ de texte gagne le focus.
  - ❖ `public void focusLost(FocusEvent e)` : évènement se produit si le champ de texte perd le focus.
- Evènements de catégories **KeyEvent** (touche clavier) : attacher un écouteur qui implémente l'interface **KeyListener** qui comporte deux méthodes abstraites :
  - ❖ `public void keyPressed(KeyEvent e)` : évènement se produit lors de l'appui sur une touche du clavier.
    - `ctxt.getText()` : contient le texte présent dans le `JTextField` avant l'appui sur la touche.
    - `e.getKeyChar()` : contient le caractère correspondant à la touche pressée
  - ❖ `public void keyReleased(KeyEvent e)` : évènement se produit après l'appui sur la touche du clavier.

## 5. Les champs texte – JTextField

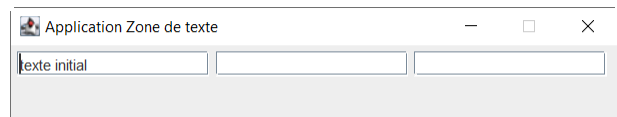
### Evènement sur les champs de texte :

- Evènement de catégories **Action** : évènement déclenché si l'utilisateur appuie sur la touche Entrée. Attacher un écouteur qui implémente l'interface **ActionListener** qui comporte la méthode abstraite suivante :
  - ❖ `public void actionPerformed(ActionEvent e)` : évènement se produit après l'appui sur la touche Entrée du clavier.
- **Comment créer un écouteur (Listener) :**
  1. Classe anonyme
  2. Classe interne
  3. Implémenter l'interface directement dans la classe courante.
  4. Expression Lambda

## 5. Les champs texte – JTextField

```
class MonApp {
    JTextField tx1, tx2, tx3;
    public MonApp() {
        JFrame fen = new JFrame("Application Etiquette");
        fen.setSize(500, 100);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        FlowLayout fl = new FlowLayout(10);
        JPanel pan = new JPanel();
        pan.setLayout(fl);
        tx1 = new JTextField("texte initial",15);
        tx2 = new JTextField("",15);
        tx3 = new JTextField("",15);
        pan.add(tx1); pan.add(tx2); pan.add(tx3);
        fen.add(pan);
        fen.setLocationRelativeTo(null);
        fen.setResizable(false);
        fen.setVisible(true);
    }
}

public class ZoneTexte {
    public static void main(String[] pm) {
        MonApp app = new MonApp();
    }
}
```



## 5. Les champs texte – JTextField

- a. Créer un écouteur avec une classe anonyme

```
...
montxt.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
        nb++;
        label.setText(String.valueOf(nb));
    }
    @Override
    public void keyReleased(KeyEvent e) {}
    @Override
    public void keyPressed(KeyEvent e) {}
});
...
```

## 5. Les champs texte – JTextField

- b. Créer un écouteur avec une classe interne

```
...
montxt.addKeyListener(new MyEventKey());
}

class MyEventKey implements KeyListener {
    public void keyTyped(KeyEvent e) {
        nb++;
        label.setText(String.valueOf(nb));
    }
    @Override
    public void keyPressed(KeyEvent e) {}
    @Override
    public void keyReleased(KeyEvent e) {}
}
}
```

## 5. Les champs texte – JTextField

- c. Créer un écouteur avec une expression Lambda

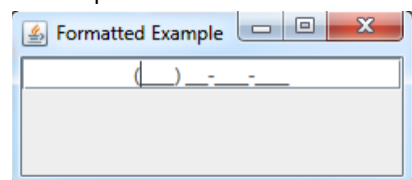
```
montxt.addActionListener((ActionListener) e-> {
    JOptionPane.showMessageDialog(null, "Vous avez tapez au total:" +nb);
});
```

Remarque :

- L'utilisation de l'expression lambda est recommandée s'il n'y a qu'une seule méthode abstraite à définir.

## 6. La classe JPasswordField

- La classe `javax.swing.JPasswordField` est un composant qui permet de saisir des caractères cachés par des étoiles.
- Les constructeurs de `JPasswordField` sont :
  - `JPasswordField()`; ➤ constructeur par défaut.
  - `JPasswordField("mon texte");` ➤ construction d'un `JPasswordField` avec un texte initial non claire.
  - `JPasswordField("mon texte", 50);` ➤ texte initial non claire et un nombre de colonnes maximum
- Les principaux méthodes sont :
  - `char[] getPassword()` : Retourne un tableau de caractères contenant le mot de passe saisi.
  - `void setEchoChar(char c)` : Change le caractère écho des caractères tapés
- On peut utiliser la classe **JFormattedTextField** pour créer des champs de texte formaté selon le besoin de l'utilisateur.



## 7. La classe JTextArea

- La classe `javax.swing.JTextArea` est un composant graphique permettant de saisir un texte en plusieurs lignes.
- Les **constructeurs** de `JTextArea` sont :
  - `JTextArea(int L, int C)`; ➤ construit un espace de saisi de texte avec L lignes et C colonnes.
  - `JTextArea(String texte)`; ➤ construit un espace de saisi de texte avec texte initial.
- Les principaux **méthodes** sont :
  - `String getText()` : Retourne le texte du `JTextArea`.
  - `String getSelectedText()` : Retourne le texte sélectionné.
  - `void setText(String t)` : Remplace le texte du `JTextArea` par t.
  - `void append(String t)` : Ajoute le texte t à la fin du `JTextArea`.
  - `void replaceSlection(String t)` : Remplace le texte sélectionné du `JTextArea` par le texte t.
  - `void copy()` : Copie le texte sélectionné dans le presse-papiers.
  - `void paste()` : colle le presse-papiers à la place du texte sélectionné

## 7. La classe JTextArea

### Evènement du composant JTextArea :

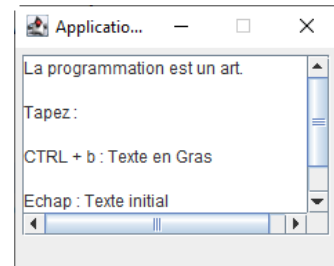
- Evènement de catégories **Key**, gérer par l'interface **KeyListener** : événement déclenché lors de l'interaction avec les touches du clavier.
- L'interface comporte trois méthodes abstraites :
  - **keyPressed(KeyEvent e)** : L'événement est déclenché lorsque une touche est enfoncée.
  - **keyReleased(KeyEvent e)** : L'événement est déclenché lorsque une touche est relâchée.
  - **keyTyped(KeyEvent e)** : L'événement est déclenché lorsque un caractère est tapé, généralement lié à des touches qui génèrent un caractère imprimable.

## 7. La classe JTextArea – Exemple

```
import java.awt.Font;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
class ta extends JFrame {
    JTextArea txta;
    JPanel pan;
    public ta() {
        super("Application texte Area");
        this.setSize(250, 200);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setLocationRelativeTo(null);
        pan = new JPanel();
        txta = new JTextArea(8, 20);
        Font oldFont = txta.getFont();
        txta.addKeyListener(new KeyListener() {
            public void keyTyped(KeyEvent e) { }
            public void keyReleased(KeyEvent e) { }
            public void keyPressed(KeyEvent e) {
                if(e.getKeyCode() == KeyEvent.VK_B && e.isControlDown()) {
                    Font newFont = oldFont.deriveFont(Font.BOLD);
                    txta.setFont(newFont);
                }
                if(e.getKeyCode() == KeyEvent.VK_ESCAPE) {
                    txta.setFont(oldFont); } } });
    }
}
```

```
JScrollPane sp = new JScrollPane();
sp.setViewportView(txta);
pan.add(sp);
add(pan);
setResizable(false);
setVisible(true);
}

public class TextAreaApp {
    public static void main(String[] args) {
        new ta();
    }
}
```



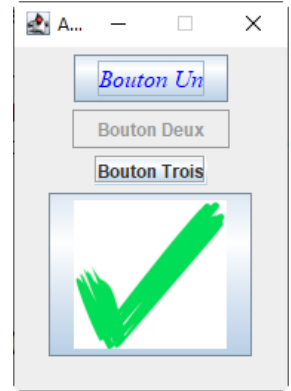
## 8. La classe JButton

- La classe javax.swing.JButton permet de créer un bouton graphique.
- Les **constructeurs** de JTextArea sont :
  - JButton(String text); ➡ créer un bouton avec du texte.
  - JButton(Icon ic); ➡ créer un bouton avec une icône.
- Les principaux **méthodes** sont :
  - setMnemonic('n'); Créer un caractère de raccourcis pour le bouton.
  - fen.getRootPane().setDefaultButton(bouton4); Activer le bouton par défaut pour la fenêtre.
  - setHorizontalAlignment(JButton.CENTER); positionner le texte du bouton au centre.
  - setFocusPainted(false); désactiver le signe du focus
  - setToolTipText("ce bouton permet de ..."); Activer l'info-bulle

## 8. La classe JButton – Exemple

```
class MesBoutons {
    public MesBoutons() {
        JFrame fen = new JFrame();
        fen.setSize(200, 270);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setTitle("Application Bouton");
        fen.setLocationRelativeTo(null);
        fen.setResizable(false);
        JPanel pan = new JPanel();
        JButton bouton1 = new JButton("Bouton Un");
        JButton bouton2 = new JButton("Bouton Deux");
        JButton bouton3 = new JButton("Bouton Trois");
        Icon ic = new ImageIcon(getClass().getResource("ok.png"));
        JButton bouton4 = new JButton(ic);
        bouton1.setFont(new Font("serif", Font.ITALIC, 16));
        bouton1.setForeground(Color.BLUE);
        bouton2.setEnabled(false);
        bouton3.setBorder(BorderFactory.createEtchedBorder());
        pan.add(bouton1); pan.add(bouton2); pan.add(bouton3);
        pan.add(bouton4); fen.add(pan); fen.setVisible(true);
    }
}

public class TestBouton {
    public static void main(String[] args) {
        new MesBoutons();
    }
}
```



## 8. La classe JButton

- Un bouton ne peut déclencher qu'un seul évènement correspond à un clic sur lui. L'évènement est de catégorie **Action** qui est géré par un écouteur implémentant l'interface **ActionListener** qui comporte la méthode abstraite `actionPerformed()`

Méthode 1 :  
Classe anonyme

```
bouton1.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(null, "Bouton 1");
    }
});
```

Méthode 2 :  
Lambda

```
bouton4.addActionListener((ActionListener) e-> {
    JOptionPane.showMessageDialog(null, "Bouton 4");
});
```

## 9. La classe JCheckBox

- La classe `javax.swing.JCheckBox` est un composant graphique permettant de créer des cases à cocher.
- Les **constructeurs** de `JCheckBox` sont :
  - `JCheckBox(String text)` ➔ case à cocher avec du texte.
  - `JCheckBox(String text, boolean b)` ➔ case à cocher avec du texte et état de sélection.
- Les principaux **méthodes** sont :
  - `isSelected()` : retourne boolean, permet de connaître l'état de la case à cocher.
  - `setSelected(boolean)` : imposer à une case à cocher un état donnée
- Les **événements** liés sont :
  - **Action** : implémenter l'interface `ActionListener` et définir la méthode abstraite `actionPerformed(ActionEvent e)`. Pour associer cet événement au composant `JCheckBox`, il faut utiliser la méthode `addActionListener()`
  - **Item** : implémenter l'interface `ItemListener` et définir la méthode abstraite `itemStateChanged(ItemEvent e)`. Pour associer un événement au composant `JCheckBox`, il faut utiliser la méthode : `addItemListener()`

## 9. La classe JCheckBox – Exemple

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;

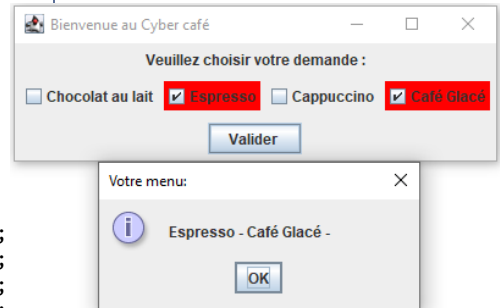
class CaseCocher {
    private JCheckBox cc1, cc2, cc3, cc4;
    private JLabel txt;
    private JButton btn;

    public CaseCocher() {
        JFrame fen = new JFrame("Bienvenue au Cybercafé");
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setSize(220, 250);
        JPanel pantxt = new JPanel();
        JPanel panCheckBox = new JPanel(new
        FlowLayout(FlowLayout.CENTER));

        JPanel panbtn = new JPanel();
        txt = new JLabel("Veuillez choisir votre demande:");
        cc1 = new JCheckBox("Chocolat au lait", false);
        cc2 = new JCheckBox("Café expresso ", false);
        cc3 = new JCheckBox("Cappuccino ", false);
        cc4 = new JCheckBox("Café Turc ", false);
        btn = new JButton("Valider ");
        pantxt.add(txt);
        panCheckBox.add(cc1);
        panCheckBox.add(cc2);
        panCheckBox.add(cc3);
        panCheckBox.add(cc4);
        panbtn.add(btn);
        Ecouteur il = new Ecouteur();
        cc1.addItemListener(il);
        cc2.addItemListener(il);
        cc3.addItemListener(il);
        cc4.addItemListener(il);
        Ecouteur2 al = new Ecouteur2();
        btn.addActionListener(al);
        fen.add(pantxt, BorderLayout.NORTH);
        fen.add(panCheckBox, BorderLayout.CENTER);
        fen.add(panbtn, BorderLayout.SOUTH);
        fen.setLocationRelativeTo(null);
    }
}
```

## 9. La classe JCheckBox – Exemple

```
fen.pack();
fen.setVisible(true);
}
private class Ecouteur implements ItemListener {
    public void itemStateChanged(ItemEvent e) {
        JCheckBox cc = (JCheckBox)e.getItem();
        if(cc.isSelected()) { cc.setBackground(Color.red);
        }else { cc.setBackground(null); }
    }
}
private class Ecouteur2 implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        StringBuilder menu = new StringBuilder();
        if (cc1.isSelected()) { menu.append(cc1.getText() + " - ");
        if (cc2.isSelected()) { menu.append(cc2.getText() + " - ");
        if (cc3.isSelected()) { menu.append(cc3.getText() + " - ");
        if (cc4.isSelected()) { menu.append(cc4.getText() + " - ");
        JOptionPane.showMessageDialog(null, menu, "Votre menu: ",
        JOptionPane.INFORMATION_MESSAGE);
    }
    }
}
public class TestCaseACocher {
    public static void main(String[] pm) {
        new CaseCocher();
    }
}
```



## 10. La classe JRadioButton

- La classe javax.swing.JRadioButton permet de créer le composant graphique bouton radio qui permet d'exprimer un choix oui/non.
- Les **constructeurs** de JRadioButton sont :
  - JRadioButton(String text) ➔ Bouton radio avec du texte.
  - JRadioButton(String text, boolean selected) ➔ Bouton radio avec du texte avec état de sélection.
  - JRadioButton(String text, Icon icon) ➔ Bouton radio avec du texte et icône
- Les principaux **méthodes** sont :
  - isSelected() : retourne boolean, permet de connaître l'état du bouton radio (sélectionné ou non).
  - setSelected(boolean) : forcer le sélectionnement du bouton radio.
  - setEnabled(boolean) : active ou désactive le bouton radio.
- **L'évènement** lié à la classe JRadioButton est :
  - **Item** : implémenter l'interface **ItemListener** et définir la méthode abstraite itemStateChanged(ItemEvent e). Pour associer un évènement au composant JRadioButton, il faut utiliser la méthode : **addItemListener()**

## 10. La classe JRadioButton

### Remarques :

- La désactivation automatique d'autres boutons radio est réalisée par l'appartenance des boutons radio dans un même groupe :

```
ButtonGroup group = new ButtonGroup()
```

- Associer chacun des boutons dans ce groupe à l'aide de la méthode add :

```
group.add(rb1);
```

```
group.add(rb2);
```

```
Groupe.add(rb3);
```

## 10. La classe JRadioButton – Exemple

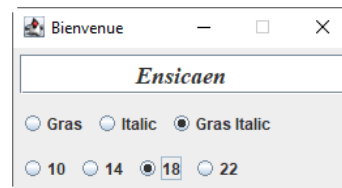
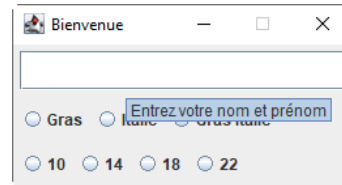
```
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import javax.swing.ButtonGroup;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JTextField;

class BoutonRadio {
    private JRadioButton rb1, rb2, rb3, rb4, rb5, rb6, rb7;
    private ButtonGroup group1, group2;
    private JTextField text;
    public BoutonRadio() {
        JFrame fen = new JFrame("Bienvenue");
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setSize(300, 200);
        JPanel pan1 = new JPanel(new FlowLayout(FlowLayout.LEFT));
        JPanel pan2 = new JPanel(new FlowLayout(FlowLayout.LEFT));
        JPanel pan3 = new JPanel(new FlowLayout(FlowLayout.LEFT));
        text = new JTextField();
        text.setHorizontalAlignment(JTextField.CENTER);
        text.setPreferredSize(new Dimension(250, 30));
        text.setToolTipText("Entrez votre nom et prénom");
        rb1 = new JRadioButton("Gras");
        rb2 = new JRadioButton("Italic");
```

```
rb3 = new JRadioButton("Gras Italic");
rb4 = new JRadioButton("10");
rb5 = new JRadioButton("14");
rb6 = new JRadioButton("18");
rb7 = new JRadioButton("22");
pan1.add(text);
pan2.add(rb1);
pan2.add(rb2);
pan2.add(rb3);
pan3.add(rb4);
pan3.add(rb5);
pan3.add(rb6);
pan3.add(rb7);
group1 = new ButtonGroup();
group1.add(rb1);
group1.add(rb2);
group1.add(rb3);
group2 = new ButtonGroup();
group2.add(rb4);
group2.add(rb5);
group2.add(rb6);
group2.add(rb7);
Ecouteur il = new Ecouteur();
rb1.addItemListener(il);
rb2.addItemListener(il);
rb3.addItemListener(il);
rb4.addItemListener(il);
rb5.addItemListener(il);
```

## 10. La classe JRadioButton – Exemple

```
rb6.addItemListener(il); rb7.addItemListener(il);
fen.add(pan1, BorderLayout.NORTH);
fen.add(pan2, BorderLayout.CENTER);
fen.add(pan3, BorderLayout.SOUTH);
fen.setResizable(false);
fen.setLocationRelativeTo(null);
fen.pack();
fen.setVisible(true);
}
private class Ecouteur implements ItemListener {
    Font fo = null;
    public void itemStateChanged(ItemEvent e) {
        int style = Font.PLAIN;
        int police = 14;
        if (rb1.isSelected()) { style = Font.BOLD; }
        if (rb2.isSelected()) { style = Font.ITALIC; }
        if (rb3.isSelected()) { style = Font.ITALIC + Font.BOLD; }
        if (rb4.isSelected()) { police = 10; }
        if (rb5.isSelected()) { police = 14; }
        if (rb6.isSelected()) { police = 18; }
        if (rb7.isSelected()) { police = 22; }
        fo = new Font("Serif", style, police);
        text.setFont(fo);
    }
}
public class TestBoutonRadio {
    public static void main(String[] args) {
        new BoutonRadio();
    }
}
```



## 11. La classe JList

- La classe `javax.swing.JList` nous permet de choisir une ou plusieurs valeurs dans une liste prédéfinie.
- Les **constructeurs** de `JList` sont :
  - `JList()` ➔ Crée une liste vide avec un modèle par défaut.
  - `JList(E[] listData)` ➔ Crée une liste à partir d'un tableau d'éléments de type E.
  - `JList(Vector<E> listData)` ➔ Crée une liste à partir d'un Vector.
- Les **principaux méthodes** sont :
  - `getSelectedValue()` : permet de retourner la seule valeur sélectionnée dans la liste, sa valeur de retour est Object, il faut procéder à une conversion explicite.
  - `getSelectedValuesList()` : permet de retourner toutes les valeurs sélectionnées dans la structure List.

```
List val = list.getSelectedValuesList();
for(int i=0; i<val.size();i++) {
    System.out.println("couleur : " + val.get(i));
}
```

## 11. La classe JList

- Les **principaux méthodes** sont :

- o `getSelectedIndex()` : permet de retourner la position de la valeur sélectionnée.
- o `getSelectedIndices()` : permet de retourner un `int[]` tableau des positions des valeurs sélectionnées.
- o `isSelectionEmpty()` : retourne un boolean, permet de vérifier si la liste est sélectionnée ou non.

- L'évènement lié à la classe JList est :

- o **ListSelection** : La sélection d'un élément de la liste provoque un évènement de type ListSelection. Il faut créer un écouteur (objet) implémentant l'interface **ListSelectionListener** et définir la méthode abstraite `valueChanged()`.

**Remarque :**

- L'interface ListSelection admet une seule méthode abstraite, donc on peut utiliser l'expression lambda pour implémenter cette méthode de manière simplifiée :

```
list.addListSelectionListener(e -> {
    //Action à définir dans la méthode valueChanged
});
```

## 11. La classe JList

```
import java.awt.BorderLayout;
import java.awt.Color;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.ListSelectionModel;
import javax.swing.event.ListSelectionEvent;
import javax.swing.event.ListSelectionListener;

class BoiteListe {
    private JFrame fen;
    private JList<String> list;
    private JLabel label;
    private JScrollPane sp;
    private JPanel pan1, pan2;
    private String ini = "Couleur choisie : ";
    private String[] couleurNom = {"Rouge", "Bleu", "Vert", "Blanc"};
    private Color[] coul = {Color.RED, Color.BLUE, Color.GREEN,
        Color.WHITE};

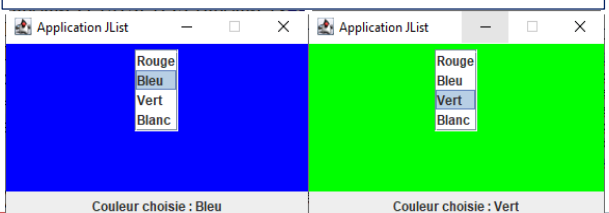
    public BoiteListe() {
        fen = new JFrame("Application JList");
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setSize(300, 200);
        fen.setLocationRelativeTo(null);
        fen.setResizable(false);
        pan1 = new JPanel();
        pan2 = new JPanel();

        list = new JList<String>(couleurNom);
        label = new JLabel(ini);
        list.setVisibleRowCount(4);
        list.setSelectionModel(ListSelectionModel.SINGLE_SELECTION);
        sp = new JScrollPane(list);
        list.addListSelectionListener(e -> {
            pan1.setBackground(coul[list.getSelectedIndex()]);
            String ch = couleurNom[list.getSelectedIndex()];
            label.setText(ini + ch);
        });
        pan1.add(sp); pan2.add(label);
        fen.add(pan1, BorderLayout.CENTER);
        fen.add(pan2, BorderLayout.SOUTH);
        fen.setVisible(true);
    }
}

public class TestBoiteListe {
    public static void main(String[] args) {
        new BoiteListe();
    }
}
```

```
list = new JList<String>(couleurNom);
label = new JLabel(ini);
list.setVisibleRowCount(4);
list.setSelectionModel(ListSelectionModel.SINGLE_SELECTION);
sp = new JScrollPane(list);
list.addListSelectionListener(e -> {
    pan1.setBackground(coul[list.getSelectedIndex()]);
    String ch = couleurNom[list.getSelectedIndex()];
    label.setText(ini + ch);
});
pan1.add(sp); pan2.add(label);
fen.add(pan1, BorderLayout.CENTER);
fen.add(pan2, BorderLayout.SOUTH);
fen.setVisible(true);
}

public class TestBoiteListe {
    public static void main(String[] args) {
        new BoiteListe();
    }
}
```



## 11. La classe JList

### Remarques :

- La classe « JScrollPane » permet de créer la barre de défilement verticale dans le composant graphique, dans notre cas la barre ne s'affiche pas. On peut afficher la barre de défilement si on diminue la valeur des lignes visibles dans le JList par setVisibleRowCount(3)
- Il existe trois types de boîtes de liste, définis par les paramètres de la méthode setSelectionMode(...)

|                                               |                                                       |
|-----------------------------------------------|-------------------------------------------------------|
| ListSelectionMode.SINGLE_SELECTION            | sélection d'une seule valeur.                         |
| ListSelectionMode.SINGLE_INTERVAL_SELECTION   | Sélection d'une plage des valeurs contiguës.          |
| ListSelectionMode.MULTIPLE_INTERVAL_SELECTION | Sélection d'un nombre quelconque de plages de valeurs |

### Exemple :

```
list.setSelectionMode(ListSelectionMode.MULTIPLE_INTERVAL_SELECTION);
```

## 12. La classe JComboBox

- La classe javax.swing.JComboBox permet de créer une liste déroulante.
- Les **constructeurs** de JComboBox sont :
  - JComboBox() ↪ Crée un Combo box vide.
  - JComboBox(String[] data) ↪ Crée une liste déroulante à partir du tableau de chaînes de caractères.
  - JComboBox (Vector<?> v) ↪ Crée un Combo box qui affiche les données contenues dans le vecteur v
- Les **principaux méthodes** sont :
  - addItem("Caen") : ajoute la ville Caen à la fin du liste.
  - addItemAt("Rouen", 2) : ajoute la ville Rouen à la position 2.
  - removeItem("Lyon") : supprime Lyon de la liste combo.
  - getItemCount() : retourne le nombre des valeurs dans la liste combo.
  - getSelectedItem() : retourne la valeur sélectionnée.
  - removeAllItems() : supprime tous les éléments de la liste combo.

## 12. La classe JComboBox

- L'évènement lié à la classe JComboBox est :
  - **Item** : La sélection d'un élément de la liste provoque un évènement de type Item. Il faut créer un écouteur qui implémente l'interface **ItemListener** et définir la méthode abstraite itemStateChanged().
  - L'interface ItemListener présente une seule méthode abstraite, donc on peut utiliser l'expression Lambda pour définir l'action à exécuter.

```
combo.addItemListener(e ->{
    label.setText((String)cb.getSelectedItem());
});
```

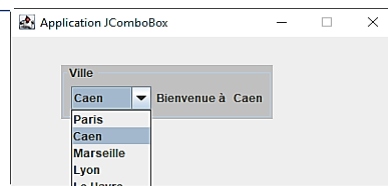
## 12. La classe JComboBox

```
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
import javax.swing.BorderFactory;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

class ComboBox extends JFrame {
    JPanel pan;
    JComboBox cb;
    JLabel lb1, lb2;
    String [] dest = {"Paris","Caen","Marseille","Lyon","Le Havre"};
    public ComboBox() {
        setTitle("Application JComboBox");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 200);
        setLocationRelativeTo(null);
        setResizable(false);
        FlowLayout fl = new FlowLayout(FlowLayout.LEFT, 50, 30);
        lb1 = new JLabel("Bienvenue à ");
        lb2 = new JLabel();
        pan = new JPanel();
        pan.setBackground(Color.LIGHT_GRAY);
        cb = new JComboBox<String>(dest);
```

```
cb.addItemListener(new ItemListener() {
    public void itemStateChanged(ItemEvent e) {
        lb2.setText((String)cb.getSelectedItem());
    }
});
// cb.addItemListener(e ->{
// lb2.setText((String)cb.getSelectedItem());
// });
pan.setBorder(BorderFactory.createTitledBorder("Ville"));
pan.add(cb);
pan.add(lb1);
pan.add(lb2);
setLayout(fl);
add(pan);
setVisible(true);
}

public class TestComboBox {
    public static void main(String[] args) {
        new ComboBox();
    }
}
```



### 13. La classe JSlider

- La classe `javax.swing.JSlider` permet de sélectionner une valeur dans une plage de valeurs numériques par un glissement.
- Les **constructeurs** de `JSlider` sont :
  - `JSlider()` ➔ créer un slider horizontal avec un intervalle entre 0 et 100 initialisé par défaut à 50.
  - `JSlider(int min, int max, int value)` ➔ créer un slider horizontal avec spécification des valeurs.
  - `JSlider(int orientation, int min, int max, int value)` ➔ créer un slider avec spécification des valeurs et de l'orientation : `JSlider.HORIZONTAL` ou `JSlider.VERTICAL`
- Les **principaux méthodes** sont :
  - `getValue()` : Retourne la valeur actuelle du curseur
  - `setValue(int value)` : Définit la valeur actuelle du curseur
  - `getMinimum()` : Retourne la valeur minimale du curseur
  - `setMinimum(int min)` : Définit la valeur minimale
  - `getMaximum()` : Retourne la valeur maximale.

### 13. La classe JSlider

- L'évènement lié à la classe `JSlider` est :
  - **ChangeEvent** : La classe `JSlider` génère des événements quand sa valeur change. Il faut créer un écouteur qui implémente l'interface **ChangeListener** et définir la méthode abstraite `stateChanged()`.
  - L'interface `ChangeListener` présente une seule méthode abstraite, donc on peut utiliser l'expression Lambda pour définir l'action à exécuter.

```
s3.addChangeListener(e -> {
    int v = s3.getValue();
    vbleu.setText(String.valueOf(v));
    c = panbg.getBackground();
    c = new Color(c.getRed(), c.getGreen(), v);
    panbg.setBackground(c);
});
```

## 13. La classe JSlider

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.FlowLayout;
import javax.swing.BorderFactory;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JSlider;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;

class MySlider extends JFrame {
    JPanel panslider, pantitre, panbg, paninfo;
    JSlider s1, s2, s3;
    JLabel rouge, vrouge, vert, vvert, bleu, vbleu;
    Color c;
    public MySlider() {
        super("Application JSlider");
        setSize(500, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        // panneau de titre
        JLabel labtitre = new JLabel("Couleur Arrière plan en RGB");
        FlowLayout fltitre = new FlowLayout(FlowLayout.CENTER);
        pantitre = new JPanel();
        pantitre.setLayout(fltitre);
        pantitre.add(labtitre);

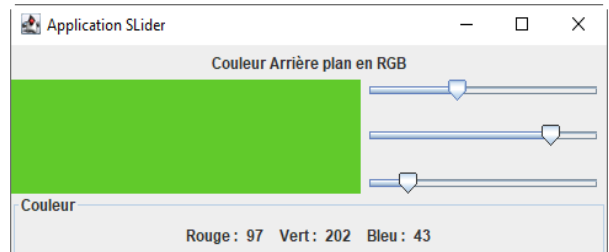
        // panneau des Sliders
        s1 = new JSlider(JSlider.HORIZONTAL, 0, 255, 200); // Rouge
        s2 = new JSlider(JSlider.HORIZONTAL, 0, 255, 100); // Vert
        s3 = new JSlider(JSlider.HORIZONTAL, 0, 255, 50); // Bleu
        s1.setToolTipText("Rouge");
        s2.setToolTipText("Vert");
        s3.setToolTipText("Bleu");
        BorderLayout blslider = new BorderLayout();
        panslider = new JPanel();
        panslider.setLayout(blslider);
        panslider.add(s1, BorderLayout.NORTH);
        panslider.add(s2, BorderLayout.CENTER);
        panslider.add(s3, BorderLayout.SOUTH);
        //panneau d'information
        rouge = new JLabel("Rouge : ");
        vrouge = new JLabel(String.valueOf(s1.getValue()));
        vert = new JLabel(" Vert : ");
        vvert = new JLabel(String.valueOf(s2.getValue()));
        bleu = new JLabel(" Bleu : ");
        vbleu = new JLabel(String.valueOf(s3.getValue()));
        FlowLayout flinfo = new FlowLayout(FlowLayout.CENTER);
        flinfo.setHgap(4);
        paninfo = new JPanel();
        paninfo.setLayout(flinfo);
        paninfo.setBorder(BorderFactory.createTitledBorder(null,
            "Couleur"));
        paninfo.add(rouge); paninfo.add(vrouge);
    }
}
```

## 13. La classe JSlider

```
paninfo.add(vert); paninfo.add(vvert);
paninfo.add(bleu); paninfo.add(vbleu);
//panneau d'arriere plan
panbg = new JPanel();
c = new Color(s1.getValue(), s2.getValue(), s3.getValue());
panbg.setBackground(c);
s1.addChangeListener(new ChangeListener() {
    public void stateChanged(ChangeEvent e) {
        int v = s1.getValue();
        vrouge.setText(String.valueOf(v));
        c = panbg.getBackground();
        c = new Color(v,c.getGreen(), c.getBlue());
        panbg.setBackground(c);
    }
});
s2.addChangeListener(new ChangeListener() {
    public void stateChanged(ChangeEvent e) {
        int v = s2.getValue();
        vvert.setText(String.valueOf(v));
        c = panbg.getBackground();
        c = new Color(c.getRed(),v , c.getBlue());
        panbg.setBackground(c);
    }
});
s3.addChangeListener(new ChangeListener() {
    public void stateChanged(ChangeEvent e) {
        int v = s3.getValue();
        vbleu.setText(String.valueOf(v));
        c = panbg.getBackground();
        c = new Color(c.getRed(),c.getGreen(), v);
    }
});

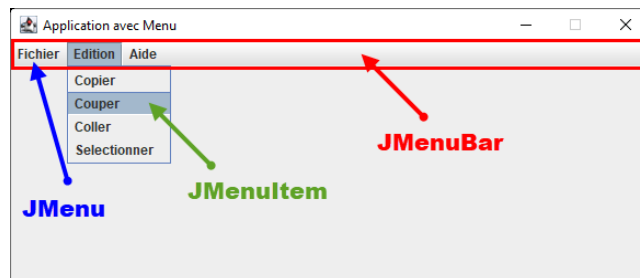
panbg.setBackground(c);
} };
```

```
public class TestJSlider {
    public static void main(String[] args) {
        new MySlider();
    }
}
```



## 14. Création de menus

- La création des menus en Java fait intervenir trois classes :
  - JMenuBar** : permet de créer une nouvelle barre de menus vide.
  - JMenu** : différents objets qui sont visible dans la barre de menus.
  - JMenuItem** : différentes options de chaque menu qui les constituent.



## 14. Création de menus

- La **création** d'un objet **barre de menu** se fait par :
 

```
JMenuBar barreMenu = new JMenuBar();
```
- Création des objets de menus :**

```
JMenu fichier = new JMenu("Fichier");
JMenu edition = new JMenu("Edition");
JMenu aide = new JMenu("Aide");
```
- Rattachement de menus** à la barre de menus selon l'ordre d'apparence :
 

```
barreMenu.add(fichier);
barreMenu.add(edition);
barreMenu.add(aide);
```

## 14. Création de menus

### ■ Création des options (items) :

```
JMenuItem copier = new JMenuItem("Copier");
JMenuItem couper = new JMenuItem("Couper");
JMenuItem coller = new JMenuItem("Coller");
```

### ■ Ajout des options (items) dans un menu selon l'ordre d'apparence :

```
edition.add(copier);
edition.add(couper);
edition.add(coller);
```

## 14. Création de menus

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.InputEvent;
import java.awt.event.KeyEvent;
import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JOptionPane;
import javax.swing.KeyStroke;

class MonMenu {
    private JMenuBar menuBar;
    private JMenu menuFichier, menuEdition, menuAide;
    private JMenuItem newf, open, close, copier, couper,
        coller, selectionner, aide, apropos;
    public MonMenu() {
        menuBar = new JMenuBar();
        //Eléments de menu
        menuFichier = new JMenu("Fichier");
        menuEdition = new JMenu("Edition");
        menuAide = new JMenu("Aide");
        menuBar.add(menuFichier);
        menuBar.add(menuEdition);
        menuBar.add(menuAide);
        //Liste de toutes les options
        newf = new JMenuItem("New");
        open = new JMenuItem("Open");
```

```
close = new JMenuItem("Close");
copier = new JMenuItem("Copier");
couper = new JMenuItem("Couper");
coller = new JMenuItem("Coller");
selectionner = new JMenuItem("Selectionner");
aide = new JMenuItem("Aide");
apropos = new JMenuItem("A propos");
//Ajout de chaque option dans son menu
menuFichier.add(newf);
menuFichier.add(open);
menuFichier.add(close);
menuEdition.add(copier);
menuEdition.add(couper);
menuEdition.add(coller);
menuEdition.add(selectionner);
menuAide.add(aide);
menuAide.add(apropos);
menuAide.setMnemonic('a');
apropos.setMnemonic('p');
open.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_O,
    InputEvent.CTRL_DOWN_MASK));
open.setToolTipText("Créer un nouveau document");
apropos.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_P,
    InputEvent.CTRL_DOWN_MASK));
copier.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(null, "Menu", "Action Copier",
            JOptionPane.INFORMATION_MESSAGE);
    }
});
```

## 14. Création de menus

```

apropos.addActionListener(e -> {
    JOptionPane.showMessageDialog(null, "Aide", "Application Swing avec Menu ", JOptionPane.INFORMATION_MESSAGE);
});
}
public JMenuBar getMenu()
{
    return menuBar;
}
}
public class TestMenu {
    public static void main(String[] args) {
        JFrame fen = new JFrame("Application avec Menu");
        MonMenu mm = new MonMenu();
        fen.setSize(600, 400);
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setLocationRelativeTo(null);
        fen.setResizable(false);
        fen.setJMenuBar(mm.getMenu());
        fen.setVisible(true);
    }
}

```

## 14. Création de menus

### ▪ Evènement :

Chaque action sur une option (*JMenuItem*) génère un évènement de type **Action** qu'on peut le traiter en associant un écouteur à l'objet correspondant.

Cet écouteur doit implémenter l'interface **ActionListener** et définir la méthode **actionPerformed()**.

```

copier.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(null, "Menu", "Action Copier ", JOptionPane.INFORMATION_MESSAGE);
    }
});

apropos.addActionListener(e -> {
    JOptionPane.showMessageDialog(null, "Aide", "Application Swing avec Menu ", JOptionPane.INFORMATION_MESSAGE);
});

```

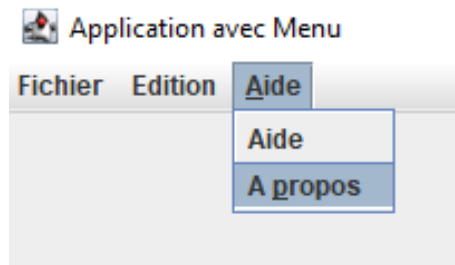
## 14. Création de menus

### Remarques :

- ✓ On peut introduire une barre séparatrice entre deux options en utilisant la méthode **addSeparator()** de la classe JMenu.
- ✓ Pour associer un caractère mnémonique à un menu ou à une option par la combinaison « Alt + caractère », on utilisant la méthode **setMnemonic()**
- ✓ Exemple :

```
menuAide.setMnemonic('a');
apropos.setMnemonic('p');
```

- ❖ Alt + a → ouvre le menu Aide
- ❖ Alt + p → lance l'option A propos



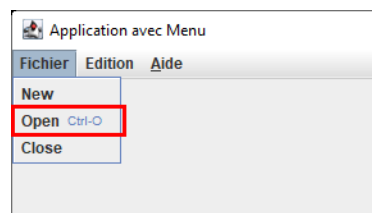
## 14. Création de menus

### Remarques :

- ✓ On peut utiliser une combinaison touches clavier pour exécuter une option par la méthode **setAccelerator()**.

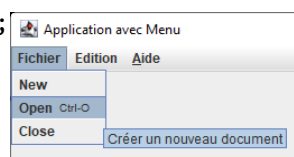
Exemple : Combinaison CTRL + O

🔗 `open.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_O, InputEvent.CTRL_DOWN_MASK));`



- ✓ On peut créer une info bulle (tooltip) sur une option par la méthode suivante :

🔗 `open.setToolTipText("Créer un nouveau document");`



## 15. Création d'un menu déroulant

- Les menus déroulants ne sont pas rattachés à la barre de menu de la fenêtre et sont affichés suite à une action de l'utilisateur, en général un clic sur le bouton droit de la souris.
- Création d'un menu surgissant :
 

```
JPopupMenu liste = new JPopupMenu();
```
- Création des items pour le menu déroulant :
 

```
JMenuItem color1 = new JMenuItem("Rouge");
JMenuItem color2 = new JMenuItem("Bleu");
```
- Ajout des items au menu déroulant :
 

```
liste.add(color1);
liste.add(color2);
```
- Un menu déroulant doit être affiché explicitement par le programme par la méthode `show`;
- ✓ Exemple :
 

```
liste.show(fen, x, y);
```

## 15. Création d'un menu déroulant

### Remarques :

- ✓ JMenuItem, génère un évènement **Action**, qui doit être traité par l'interface **ActionListener** et définir la méthode **actionPerformed()**.
- ✓ La fenêtre JFrame génère un évènement **MouseEvent**, qui doit être traité par l'interface **MouseListener** ou la classe abstraite **MouseAdapter** et définir la méthode : `mousePressed(MouseEvent e)`.
- ✓ La classe **MouseAdapter** simplifie l'implémentation de l'interface `MouseListener` en fournissant des implémentations des méthodes vides.

```
fen.addMouseListener(new MouseAdapter() {
    public void mousePressed(MouseEvent e){
        if(e.getButton()==3) {
            liste.show(fen.this, e.getX(), e.getY());
        }
    }
});
```

## 16. Les onglets

- La classe `javax.swing.JTabbedPane` permet de créer un ensemble d'onglets.
- Les **constructeurs** de `JTabbedPane` sont :
  - `JTabbedPane()` ➔ créer un groupe d'onglets avec un positionnement par défaut en haut.
  - `JTabbedPane(int tabPosition)` ➔ créer un groupe d'onglets avec un emplacement spécifique : `JTabbedPane.TOP`, `JTabbedPane.BOTTOM`, `JTabbedPane.LEFT`, ou `JTabbedPane.RIGHT`
- Les principaux méthodes sont :
  - `addTab(String, Component);`
  - `insertTab(String, Icon, Component, String, index);`
  - `removeTabAt(int index);`

## 16. Les onglets

```
import java.awt.Color;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTabbedPane;
import javax.swing.JTextField;
import javax.swing.border.Border;

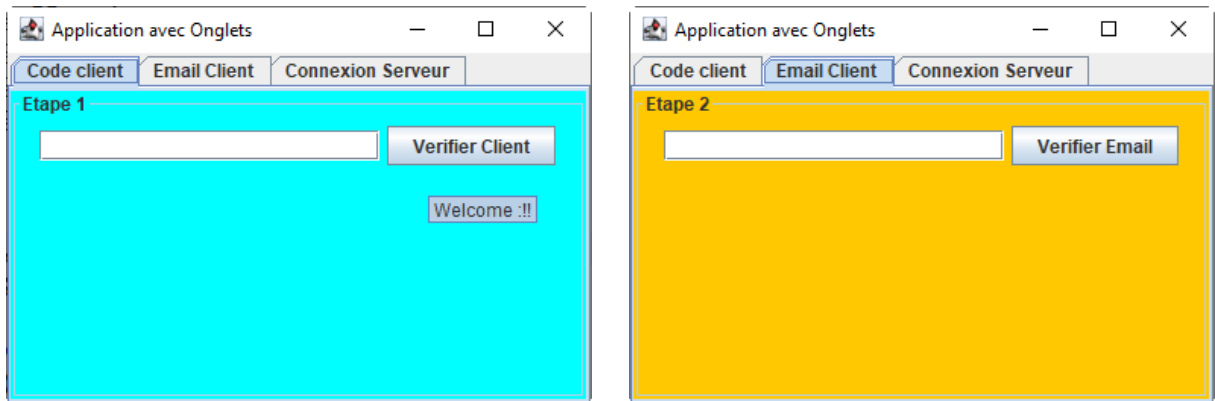
class Onglet {
    JFrame fen;
    JPanel p1, p2, p3;
    JTabbedPane o;
    JButton btn1, btn2;
    JTextField ztxt1, ztxt2;

    public Onglet() {
        fen = new JFrame("Application avec Onglets");
        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        fen.setSize(400, 270);
        fen.setLocationRelativeTo(null);
        ztxt1 = new JTextField(20);
        btn1 = new JButton("Verifier Client");
        ztxt2 = new JTextField(20);
        btn2 = new JButton("Verifier Email");
        p1 = new JPanel();
        Border b1 = BorderFactory.createTitledBorder("Etape 1");
        p1.setBorder(b1);
        p1.add(ztxt1);
```

```
        p1.add(btn1);
        p1.setBackground(Color.CYAN);
        p2 = new JPanel();
        Border b2 = BorderFactory.createTitledBorder("Etape 2");
        p2.setBorder(b2);
        p2.add(ztxt2);
        p2.add(btn2);
        p2.setBackground(Color.ORANGE);
        p3 = new JPanel();
        o = new JTabbedPane(JTabbedPane.TOP);
        o.addTab("Code client", p1);
        o.addTab("Email Client", p2);
        o.addTab("Connexion Serveur", p3);
        o.setToolTipText("Welcome !!");
        fen.add(o);
        fen.setVisible(true);
    }

    public class TestOnglet {
        public static void main(String[] args) {
            new Onglet();
        }
    }
}
```

## 16. Les onglets



## 17. La classe JDialog

- La classe `javax.swing.JDialog` permet de créer des boîtes de dialogue, qui permettent des interactions avec l'utilisateur de l'application.
- Un dialogue peut être :
  - ❖ modal : lorsqu'il est actif, toute interaction avec les autres fenêtres sont bloquées
  - ❖ non modal : ne bloque pas les interactions avec les autres fenêtres
- Les **constructeurs** de `JDialog` sont :
  - `JDialog (Frame f)` : Construit une boîte de dialogue dépendant du `Frame f` et non modale.
  - `JDialog (Frame f, String titre, boolean modal)` : Construit une boîte de dialogue dépendant du `Frame f` avec un titre et type de modalité.
- Les **principaux** méthodes sont :
  - `dispose()` : permet de fermer la boîte de dialogue et libérer l'espace mémoire utilisé.

## 17. La classe JDialog

```
import java.awt.Color;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JDialog;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.border.Border;
```

```
class JDialogGui extends JFrame {
    JMenuBar mb;
    JMenu admin;
    JMenuItem cnx, dcnx, quitter;
    JDialog diag;
    JPanel p1;
    JLabel res;
```

```
public JDialogGui() {
    setUndecorated(true);
    setSize(400, 300);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setJMenuBar(doMenu());
    diag = new JDialog(this, "Connexion...", true);
    diag.setSize(300, 250);
    diag.setLocationRelativeTo(this);
    diag.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
    diag.add(getPanneau());
    res = new JLabel();
    add(res);
    setVisible(true);
}

private JMenuBar doMenu() {
    mb = new JMenuBar();
    admin = new JMenu("Administration");
    cnx = new JMenuItem("Connexion");
    dcnx = new JMenuItem("Deconnexion");
    quitter = new JMenuItem("Quitter");
    admin.add(cnx);
    admin.add(dcnx);
    admin.add(quitter);
    mb.add(admin);
    cnx.addActionListener(e->{ diag.setVisible(true);});
    quitter.addActionListener(e->{System.exit(0);});
    return mb;
}
```

Ensicaen©2025 - HM

69

## 17. La classe JDialog

```
private JPanel getPanneau() {
    JPanel pcnx;
    JButton cplus, annuler;
    JTextField login;
    JPasswordField password;
    JLabel llogin, lpassword;
    pcnx = new JPanel(new FlowLayout(FlowLayout.CENTER));
    llogin = new JLabel("Login : ");
    lpassword = new JLabel("Password : ");
    login = new JTextField(17);
    password = new JPasswordField(17);
    cplus = new JButton("Connecter");
    annuler = new JButton("Annuler !");
    annuler.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            diag.dispose(); } });
    cplus.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            char [] pass1 = password.getPassword();
            String pass2 = String.valueOf(pass1);
            if (login.getText().equals("") || pass2.equals("")) {
                JOptionPane.showMessageDialog(null, "Champs vide",
                    "Erreur", JOptionPane.ERROR_MESSAGE);
            } else if (login.getText().equals("admin") ||
                pass2.equals("1234")) {
                JOptionPane.showMessageDialog(null,
                    "Authentification avec succès", "Authentification !",
                    JOptionPane.PLAIN_MESSAGE);
```

```
diag.dispose();
res.setFont(new Font("serif", Font.BOLD, 32));
res.setBackground(Color.LIGHT_GRAY);
res.setText(" Bonjour Admin!");
} else {
    JOptionPane.showMessageDialog(null, "Informations non
        valide", "Erreur", JOptionPane.ERROR_MESSAGE);
}
});
pcnx.add(llogin);
pcnx.add(login);
pcnx.add(lpassword);
pcnx.add(password);
pcnx.add(cplus);
pcnx.add(annuler);
Border borderClasse =
    BorderFactory.createTitledBorder("Authentification :");
pcnx.setBorder(borderClasse);
return pcnx;
}

public class _94_AppJDialog {
    public static void main(String[] args) {
        new JDialogGui();
    }
}
```

Ensicaen©2025 - HM

70

## 18. Les boîtes de dialogues « JOptionPane »

- La classe `javax.swing.JOptionPane` permet de créer plusieurs types de boîtes de dialogues modaux pour assurer la communication homme machine.

a. **Boîte de dialogue d'information** : La méthode `showMessageDialog` permet d'afficher un message à l'utilisateur.

### Syntaxe :

```
JOptionPane.showMessageDialog(Parent, "message", "titre", messageType);
```

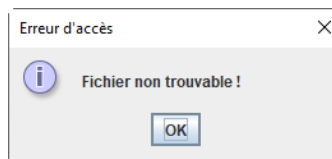
Le type de message est l'une des valeurs suivantes :

- `JOptionPane.PLAIN_MESSAGE;`
- `JOptionPane.ERROR_MESSAGE;`
- `JOptionPane.INFORMATION_MESSAGE;`
- `JOptionPane.WARNING_MESSAGE;`
- `JOptionPane.QUESTION_MESSAGE;`

## 18. Les boîtes de dialogues « JOptionPane »

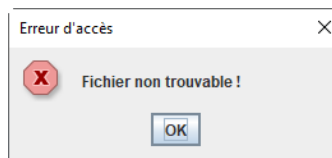
### ❖ Exemple Boîte de dialogue avec message d'information :

```
JOptionPane.showMessageDialog(null, "Fichier non trouvable !", "Erreur d'accès", JOptionPane.INFORMATION_MESSAGE);
```



### ❖ Exemple Boîte de dialogue avec message d'erreur :

```
JOptionPane.showMessageDialog(null, "Fichier non trouvable !", "Erreur d'accès", JOptionPane.ERROR_MESSAGE);
```



## 18. Les boîtes de dialogues « JOptionPane »

- b. Boîte de dialogue de confirmation :** La méthode **showConfirmDialog** permet de faire apparaître un message, avec une demande de confirmation.

### Syntaxe :

```
JOptionPane.showConfirmDialog(Parent, "message", "titre", messageType);
```

L'options de messageType est l'une des valeurs suivantes :

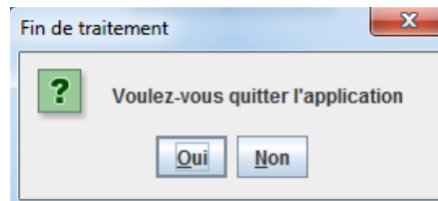
- JOptionPane.DEFAULT\_OPTION;
- JOptionPane.YES\_NO\_OPTION;
- JOptionPane.YES\_NO\_CANCEL\_OPTION;
- JOptionPane.OK\_CANCEL\_OPTION;

## 18. Les boîtes de dialogues « JOptionPane »

### ❖ Exemple Boîte de dialogue avec confirmation Oui/Non :

```
int val = JOptionPane.showConfirmDialog(null, "Voulez-vous quitter l'application", "Fin de traitement",
JOptionPane.YES_NO_OPTION);
```

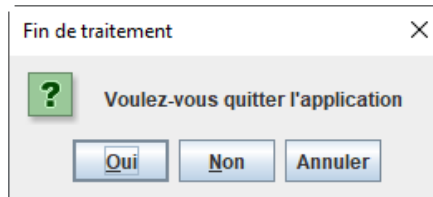
- val = 0 , si l'utilisateur clique sur Oui
- val = 1 , si l'utilisateur clique sur Non



### ❖ Exemple Boîte de dialogue avec confirmation Oui/Non/Annuler :

```
int val = JOptionPane.showConfirmDialog(null, "Voulez-vous quitter l'application", "Fin de traitement",
JOptionPane.YES_NO_CANCEL_OPTION);
```

- val = 0 , si l'utilisateur clique sur Oui
- val = 1 , si l'utilisateur clique sur Non
- val = 2 , si l'utilisateur clique sur Annuler



## 18. Les boîtes de dialogues « JOptionPane »

- c. **Boîte de dialogue de saisie** : L'appel de la méthode **showInputDialog** de la classe **JOptionPane** permet de faire une saisie de chaîne de caractères.

### Syntaxe :

```
JOptionPane.showInputDialog(Parent, "message", "titre", messageType);
```

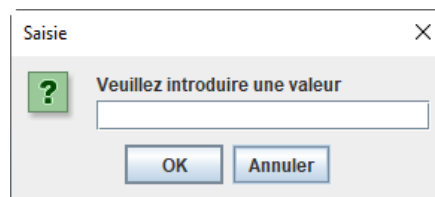
L'option de messageType est l'une des valeurs suivantes :

- JOptionPane.ERROR\_MESSAGE;
- JOptionPane.INFORMATION\_MESSAGE;
- JOptionPane.WARNING\_MESSAGE;
- JOptionPane.QUESTION\_MESSAGE;
- JOptionPane.PLAIN\_MESSAGE;

## 18. Les boîtes de dialogues « JOptionPane »

### ❖ Exemple 1 : Boîte de dialogue de saisie – entrée d'une valeur :

```
String retour = JOptionPane.showInputDialog(null, "Veuillez introduire une valeur", "Saisie", JOptionPane.QUESTION_MESSAGE);
```



### ❖ Exemple 2 : Boîte de dialogue de saisie – sélection d'une valeur :

```
String [] choix = {"Chocolat", "Fraise", "Citron", "Vanille"};
```

```
String resultat = (String) JOptionPane.showInputDialog(null, "Sélectionner une valeur", "Choisir",  
JOptionPane.QUESTION_MESSAGE, null, choix, choix[0]);
```



## 19. La classe JColorChooser

- La classe JColorChooser est un composant graphique qui permet de choisir une couleur à partir de différent modèle.

### Les constructeurs :

On peut créer un objet de ce composant par :

- ✓ JColorChooser() : Créer un composant de choix de couleur initialisé à la couleur blanche.
- ✓ JColorChooser(Color c) : Créer un composant de choix de couleur initialisé à la couleur c.

### Les principaux méthodes :

- ✓ Color showDialog(Component parent, String title, Color initialColor). //méthode statique
- ✓ Color getColor() : Retourne la couleur actuellement sélectionnée.
- ✓ void setColor(Color color) : Définit la couleur actuellement sélectionnée dans le sélecteur.

## 19. La classe JColorChooser

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JColorChooser;
import javax.swing.JFrame;
import javax.swing.JPanel;

class GuiCouleur extends JFrame implements ActionListener {
    private JPanel phaut, pbas, pcentre;
    private JButton btnHaut, btnBas, btnCentre;
    public GuiCouleur() {
        super("Choisir vos couleurs");
        setSize(500, 150);
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        phaut = new JPanel();
        pbas = new JPanel();
        pcentre = new JPanel();
        phaut.setBackground(Color.BLUE);
        pbas.setBackground(Color.CYAN);
        pcentre.setBackground(Color.DARK_GRAY);
        btnHaut = new JButton("Changer la couleur");
        btnBas = new JButton("Changer la couleur");
        btnCentre = new JButton("Changer la couleur");
        btnHaut.addActionListener(this);
```

```
btnCentre.addActionListener(this);
btnBas.addActionListener(this);
phaut.add(btnHaut);
pcentre.add(btnCentre);
pbas.add(btnBas);
add(phaut, BorderLayout.NORTH);
add(pbas, BorderLayout.SOUTH);
add(pcentre, BorderLayout.CENTER);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    Object o = e.getSource(); Color c;
    if (o == btnHaut) {
        c = phaut.getBackground();
        c = JColorChooser.showDialog(null, "choisir votre couleur", c);
        phaut.setBackground(c); }
    if (o == btnCentre) {
        c = pcentre.getBackground();
        c = JColorChooser.showDialog(null, "choisir votre couleur", c);
        pcentre.setBackground(c); }
    if (o == btnBas) {
        c = pbas.getBackground();
        c = JColorChooser.showDialog(null, "choisir votre couleur", c);
        pbas.setBackground(c); } }

public class _96_TestColorChooser {
    public static void main(String[] args) {
        new GuiCouleur();
    } }
```

## 20. Gestion de clic dans la fenêtre

- Les événements de la souris dans la fenêtre sont gérés par l'interface **MouseListener**.
- L'interface **MouseListener** comporte cinq méthodes correspondant chacune à un événement particulier :
  - **mousePressed** : Déclenché quand le bouton de la souris est enfoncé.
  - **mouseClicked** : Déclenché quand le bouton de la souris est relâché.
  - **mouseReleased** : Déclenché quand un clic complet a eu lieu (pression + relâchement).
  - **mouseEntered** : Déclenché quand le curseur entre dans la zone d'un composant.
  - **mouseExited** : Déclenché quand le curseur sort de la zone d'un composant.

## 20. Gestion de clic dans la fenêtre

```
class EcouteurSouris implements MouseListener {
    public void mouseClicked(MouseEvent event) { ... }
    public void mouseReleased(MouseEvent event) { ... }
    public void mousePressed(MouseEvent event) { ... }
    public void mouseEntered(MouseEvent event) { ... }
    public void mouseExited(MouseEvent event) { ... }
}
```

➞ Pour ajouter cet écouteur à la fenêtre : `fen.addMouseListener(new EcouteurSouris());`

- L'interface **MouseListener** nous oblige d'implémenter les cinq méthodes, même avec une implémentation vide.

## 20. Gestion de clic dans la fenêtre

- Java dispose d'une classe particulière **MouseAdapter** qui implémente toute les méthodes de l'interface **MouseListener** avec un corps vide.
- On peut donc facilement créer une classe écouteur pour les événements souris qui ne comporte qu'une méthode voulu (par exemple `mouseClicked`) comme suit :

```
class EcouteurSouris extends MouseAdapter {
    public void mouseClicked(MouseEvent event) {
        ...
    }
}
```

- On peut utiliser la notion de classe anonyme comme suit :

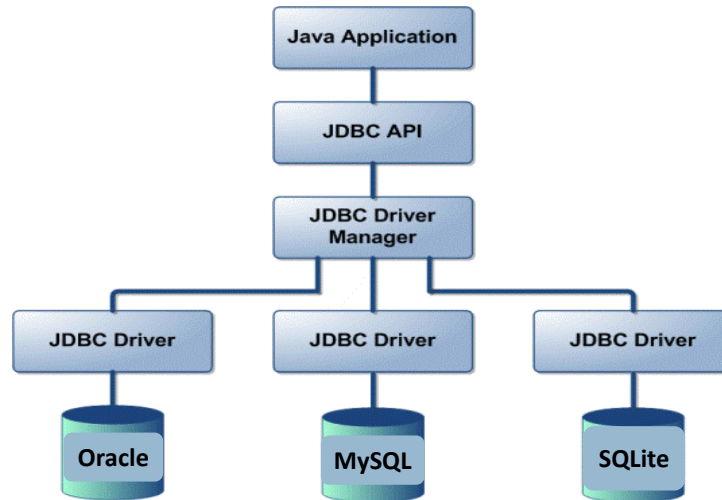
```
class maFenetre extends JFrame {
    ... ..
    addMouseListener(new MouseAdapter() {
        public void mouseClicked(MouseEvent ev) {
            ... ..
        }
    });
}
```

## V. Connexion à la base de données avec JDBC

- JDBC (**J**ava **D**ata**B**ase **C**onnectivity) est une API Java qui permet de se connecter à une base de données relationnelle, d'exécuter des requêtes SQL et de traiter les résultats retournés.
- JDBC est compatible avec la plupart des SGBD (MySQL, PostgreSQL, Oracle, SQL Server, etc.).
- Permet d'exécuter directement des requêtes SQL sans couche supplémentaire.
- Contrôle des transactions avec gestion des opérations de *commit* et de *rollback*.

## V. Connexion à la base de données avec JDBC

- Architecture générale du JDBC :



## V. Connexion à la base de données avec JDBC

- Architecture à 2 niveaux avec JDBC :



- Architecture à 3 niveaux avec JDBC :



## V. Connexion à la base de données avec JDBC

### Types de drivers JDBC :

- **Driver JDBC-ODBC (Type 1)** : utilise le pont ODBC pour accéder à la base de données. Peu utilisé aujourd'hui car lent et dépendant de la plateforme.
- **Driver natif (Type 2)** : utilise une bibliothèque native (généralement écrite en C/C++) propre au SGBD. Nécessite l'installation de bibliothèques natives.
- **Driver pont vers middleware (Type 3)** : communique avec un serveur intermédiaire (middleware) qui à son tour se connecte à la base de données. Il est indépendant du SGBD côté client mais nécessite un serveur intermédiaire.
- **Driver 100% Java (Type 4)** : communique directement avec la base de données via le protocole natif du SGBD. Il est indépendant de la plateforme et ne nécessite aucune installation supplémentaire. C'est le plus utilisé aujourd'hui.  
exemple : `com.mysql.cj.jdbc.Driver` pour MySQL.

## V. Connexion à la base de données avec JDBC

### **Package `java.sql`**

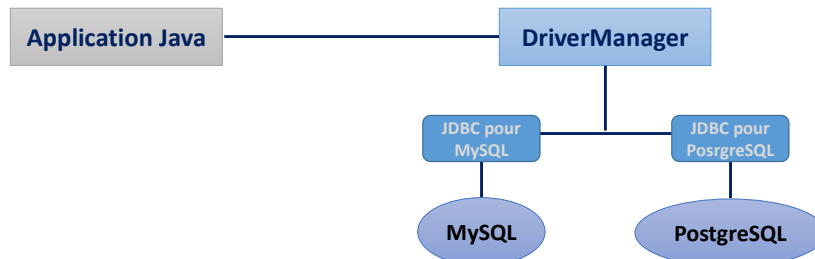
Les classes essentielles pour travailler avec JDBC sont regroupées dans le package **`java.sql`**.

Ce package contient :

- **DriverManager** : pour gérer les drivers JDBC.
- **Connection** : représente la connexion à une base de données.
- **Statement, PreparedStatement** : pour exécuter des requêtes SQL.
- **ResultSet** : pour parcourir les résultats d'une requête.
- **SQLException** : pour la gestion des erreurs liées au SQL.

## V. Connexion à la base de données avec JDBC

### Connexion à la base de données :



**DriverManager** est une classe utilitaire de l'API JDBC, permet de :

- Sélectionner le driver approprié en fonction de l'**URL** de connexion fournie (chaîne de connexion).
- Établir une connexion entre l'application Java et la base de données.

## V. Connexion à la base de données avec JDBC

### **Étapes de connexion à la base de données**

**1°) Télécharger et ajouter le driver JDBC spécifique** : Le driver JDBC est un **fichier JAR** contenant les classes nécessaires pour que l'application Java se connecte à un SGBD.

| Base de données | Driver JDBC                     | Fichier Jar (à télécharger)   |
|-----------------|---------------------------------|-------------------------------|
| Oracle          | oracle.jdbc.driver.OracleDriver | ojdbc8.jar ou ojdbc11.jar     |
| MySQL           | com.mysql.cj.jdbc.Driver        | mysql-connector-j-x.x.x.jar   |
| PostgreSQL      | org.postgresql.Driver           | postgresql-x.x.x.jar          |
| SQLite          | org.sqlite.JDBC                 | sqlite-jdbc-x.x.x.jar         |
| MariaDB         | org.mariadb.jdbc.Driver         | mariadb-java-client-x.x.x.jar |

## V. Connexion à la base de données avec JDBC

### Etapes de connexion à la base de données

2°) **Ajouter le driver à IDE** : pour que Java puisse accéder et utiliser le driver JDBC (fichier JAR), il faut ajouter ce fichier dans le **classpath**, (liste de répertoires / fichiers JAR que Java utilise pour chercher des classes pour exécuter l'application).

#### a. Ajout du driver JDBC MySQL / PostgreSQL dans un projet Java sous Eclipse

- ❖ Télécharger le driver JDBC de **MySQL** : mysql-connector-j-9.1.0.jar <https://dev.mysql.com/downloads/connector/j/>
- ❖ Télécharger le driver JDBC de **PostgreSQL** : postgresql-42.7.5.jar <https://jdbc.postgresql.org/download/>
- ❖ Dans Eclipse, clique droit sur le nom du projet → Properties → Java Build Path → Libraries → ClassPath
- ❖ Cliquer sur Add External JARs
- ❖ Sélectionner le fichier driver .jar
- ❖ Cliquer sur Apply and Close

## V. Connexion à la base de données avec JDBC

### Etapes de connexion à la base de données

#### a. Ajout du driver JDBC MySQL dans un projet Java sous Eclipse

The screenshot shows the 'Properties for Chapitre9' dialog box in Eclipse, specifically the 'Java Build Path' tab. The 'Libraries' section is expanded, showing the 'Classpath' list. The 'Add External JARs...' button is highlighted with a red box. The 'Classpath' list contains the following entries:

- JRE System Library [JavaSE-17]
- mysql-connector-j-9.1.0.jar - D:\Softwares\01. Developpement\Driver database\MySQL\mysql-connector-j-9.1.0.jar
- postgresql-42.7.5.jar - D:\Softwares\01. Developpement\Driver database\PostgreSQL\postgresql-42.7.5.jar

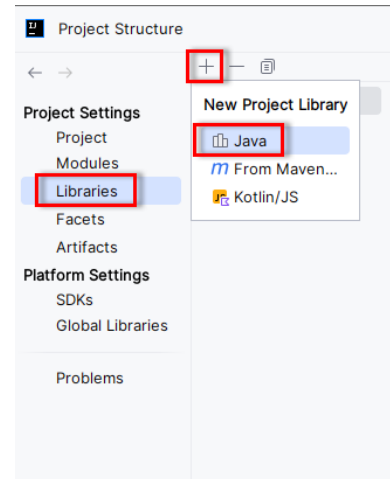
The 'Apply and Close' button is visible at the bottom right of the dialog.

## V. Connexion à la base de données avec JDBC

### Étapes de connexion à la base de données

#### b. Ajout du driver JDBC MySQL dans un projet Java sous IntelliJ

- ❖ Cliquez droit sur le projet → Open Module Settings
- ❖ Cliquez sur Libraries puis sur +
- ❖ Sélectionner le fichier driver



## V. Connexion à la base de données avec JDBC

### Étapes de connexion à la base de données

#### 3°) Chargement du driver :

- `Class.forName("Driver")` : permet de charger et initialiser le pilote de connexion à la BD.
  - Driver pour MySQL : `com.mysql.cj.jdbc.Driver`
  - Driver pour PostgreSQL : `org.postgresql.Driver`
- L'appel de la méthode `forName()` peut lever une exception de type `ClassNotFoundException`.

#### 4°) Obtention d'une connexion à la base de données :

- La classe `DriverManager` est responsable de la gestion des pilotes JDBC.
- La méthode `getConnection(url, user, password)` permet d'obtenir une connexion à la source de données.

```
Connection cnx = DriverManager.getConnection(url, user, password);
```

- ❖ url : chaîne de connexion à la BD.
  - url pour MySQL : `jdbc:mysql://localhost:3306/mydatabase`
  - url pour PostgreSQL : `jdbc:postgresql://localhost:5432/mydatabase`
- ❖ user / password : nom d'utilisateur et son mot de passe créée dans le SGBD.

## V. Connexion à la base de données avec JDBC

### Etapes de connexion à la base de données

#### 5°) Création d'un Statement

- Un Statement permet d'exécuter des requêtes SQL sur la base de données.
- La méthode **createStatement()** de l'objet **Connection** permet de créer un Statement

```
Statement stm = cnx.createStatement();
```

#### 6°) Exécution d'une requête SQL

- Pour une requête de type sélection, on utilise la méthode : **executeQuery()**

```
ResultSet rs = stm.executeQuery(sql);
```

- Pour les autres type de requête (insert, update, delete), on utilise la méthode : **executeUpdate()**

```
int ligne = stm.executeUpdate(sql);
```

❖ **ligne** : nombre des lignes affectées

## V. Connexion à la base de données avec JDBC

### Etapes de connexion à la base de données

#### 6°) Exécution d'une requête SQL

- **PreparedStatement** est une interface de l'API JDBC qui permet d'exécuter des requêtes SQL paramétrées (contenant des "?" à la place de valeurs fixes).
- PreparedStatement protège contre les attaques par injection SQL et améliore les performances à l'exécution.

Exemple :

```
PreparedStatement req = cnx.prepareStatement("SELECT * FROM user WHERE matricule = ? AND password = ?");
req.setString(1, mat);
req.setString(2, mdp);
rs = req.executeQuery();
```

## V. Connexion à la base de données avec JDBC

### Etapes de connexion à la base de données

#### 7) Affichage des résultats

- La méthode **next()** d'un objet **ResultSet** permet de parcourir ligne par ligne le résultat d'une requête SQL de sélection.

```
while(rs.next()) {
    int val1 = rs.getInt("nom_colonne");
    String val2 = rs.getString(numéro_colonne);
    System.out.println(val1, val2);
}
```

## V. Connexion à la base de données avec JDBC – Exemple 1

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

class ConnexionMySQL {
    Connection cnx = null;
    Statement stm = null;
    ResultSet rs = null;
    String url, user, password;
    public ConnexionMySQL() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
        url = "jdbc:mysql://localhost:3306/mydatabase";
        user = "root";
        password = "toon";
        try {
            cnx = DriverManager.getConnection(url,user,password);
            System.out.println("Connexion établie");
            stm = cnx.createStatement();
            String sql = "select * from voitures";
            rs = stm.executeQuery(sql);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
while(rs.next()) {
    // int code = rs.getInt("id");
    int code = rs.getInt(1);
    String nomv = rs.getString("nomvoiture");
    int p = rs.getInt(3);
    String coul = rs.getString(4);
    System.out.println("Voiture: " + nomv +
        " Puissance: " + p +
        " Couleur: " + coul);
    String sql2 = "INSERT INTO voitures VALUES (700,
        'Toyota', 5, 50000, 'Gris')";
    int val = stm.executeUpdate(sql2);
    if(val!=0) {
        System.out.println("voiture ajoutée!");
    }
    String sql3 = "DELETE FROM voitures where id = 700;";
    int val2 = stm.executeUpdate(sql3);
    if(val2!=0) {
        System.out.println("Voiture supprimée !");
    }
} catch (SQLException e) { e.printStackTrace(); }
}}

public class TestCnxMySQL {
    public static void main(String[] args) {
        new ConnexionMySQL();
    }
}
```

## V. Connexion à la base de données avec JDBC – Exemple 2

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

class maConnexion2 {
    Connection cnx = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;
    String url, user, password;
    public maConnexion2() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
    url = "jdbc:mysql://localhost:3306/mydatabase";
    user = "root";
    password = "toon";
    try {
        cnx = DriverManager.getConnection(url,user,password);
        System.out.println("Connection etablie");
        String sql = "Select * from voitures where puissance > ? and couleur = ?";

        pstmt = cnx.prepareStatement(sql);
        pstmt.setInt(1, 8);
        pstmt.setString(2, "White");
        rs = pstmt.executeQuery();
        while(rs.next()) {
            System.out.println("Voiture : " + rs.getString(2) + " - "
                + "Couleur : " + rs.getString(5));
        }
    } catch (SQLException e) { e.printStackTrace(); }
    }

    public class TestConnexion2 {
        public static void main(String[] args) {
            new maConnexion2();
        }
    }
}
```

## V. Connexion à la base de données avec JDBC – MySQL

MySQL Workbench

Local instance MySQL92 - W... x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

mydatabase

Tables

voitures

Views

Stored Procedures

Functions

sakila

Administration Schemas

Information

Table: voitures

Columns:

id int AI PK

nomvoiture varchar(45)

puissance int

kilometrage int

couleur varchar(45)

voitures x

1 • SELECT \* FROM mydatabase.voitures;

Result Grid

| id   | nomvoiture  | puissance | kilometrage | couleur |
|------|-------------|-----------|-------------|---------|
| 100  | Peugeot 508 | 7         | 54000       | Red     |
| 200  | Citroen     | 5         | 78000       | Green   |
| 300  | BMW         | 11        | 65000       | Black   |
| 400  | Mercedes    | 12        | 36000       | White   |
| 500  | Hyundai     | 6         | 85000       | Grey    |
| 600  | Jaguar      | 13        | 75000       | White   |
| NULL | NULL        | NULL      | NULL        | NULL    |

## V. Connexion à la base de données avec JDBC – PostgreSQL

pgAdmin 4

File Object Tools Edit View Window Help

Welcome public.voitures/mydatabase/postgres@PostgreSQL 17 X

public.voitures/mydatabase/postgres@PostgreSQL 17

No limit

Query Query History

```
1 SELECT * FROM public.voitures
2 ORDER BY id ASC
```

Data Output Messages Notifications

|   | id<br>[PK] integer | nomvoiture<br>character varying (10) | couleur<br>character varying (7) | puissance<br>integer |
|---|--------------------|--------------------------------------|----------------------------------|----------------------|
| 1 | 1                  | Clio                                 | Rouge                            | 5                    |
| 2 | 2                  | Golf                                 | Noir                             | 7                    |
| 3 | 3                  | BMW                                  | Bleu                             | 12                   |
| 4 | 4                  | Megane                               | Gris                             | 6                    |
| 5 | 5                  | Hyundai                              | Jaune                            | 7                    |
| 6 | 6                  | Mercedes                             | Noir                             | 12                   |

Ensicaen©2025 - HM 99

## V. Connexion à la base de données avec JDBC

### 8°) Java et les transactions

- Par défaut, les opérations sur la base de données sont en mode **auto-commit**.
- Dans ce mode, chaque opération est validée unitairement pour former la transaction.
- Pour rassembler plusieurs opérations en une seule transaction:
  - `connection.setAutoCommit(false);`
  - `connection.commit();`
- Pour annuler une transaction:
  - `connection.rollback();`